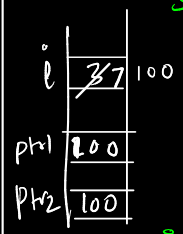
→ A pointer can point to different variables at different points in program. e.g. ptr points to i and j

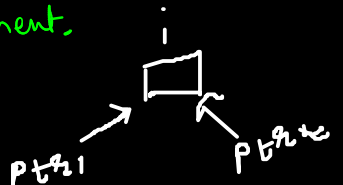
→ But at any given point in program, a pointer can point to only one variable. e.g. ptr can not point to i and j at the same time.

```
int i=3, j=5;
int *ptr = &i;
printf("%d", *ptr); //3
ptr = &j;
printf("%d", *ptr); //5
```

→ Multiple pointers can point to the same variable (even at the same time).

e.g. Both ptr1 & ptr2 are pointing to i. change in value of i to 7, results in output 7 7 7 for second printf statement.





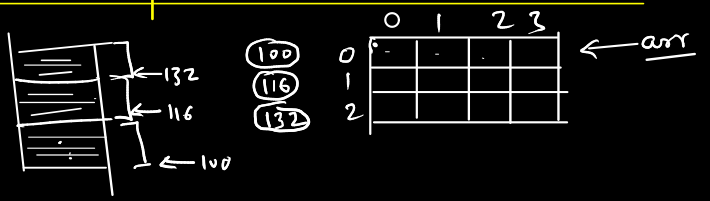

```
int i=3, *ptr1=&i, *ptr2=&i;
printf("%d %d %d", i, *ptr1, *ptr2);
//3 3 3
→ *ptr1=7;
printf("%d %d %d", i, *ptr1, *ptr2);
//7 7 7
```

→ Never leave pointers uninitialized.
Because dereferencing pointer, which is holding garbage value, may lead to segmentation fault or unexpected behaviour.

```
int *ptr;        // (NULL)
*ptr = 7; // Segmentation fault ✓
          // or Unexpected behaviour.
```

`int arr[3][4];` (like constant)

arr	pointer to first row.
arr+i	pointer to i^{th} row.
*(arr+i) arr[i]	pointer to first element of i^{th} row.
*(arr+i)+j arr[i]+j	pointer to j^{th} element of i^{th} row.
((arr+i)+j) arr[i][j] *(arr[i]+j) (*(arr+i))[j]	Value of j^{th} element of i^{th} row.



✡ Special case for string

```
int main(){
    char *ptr = "Ganga";
    printf("%s", ptr); // Ganga
```

✗ `scanf("%s", ptr);`

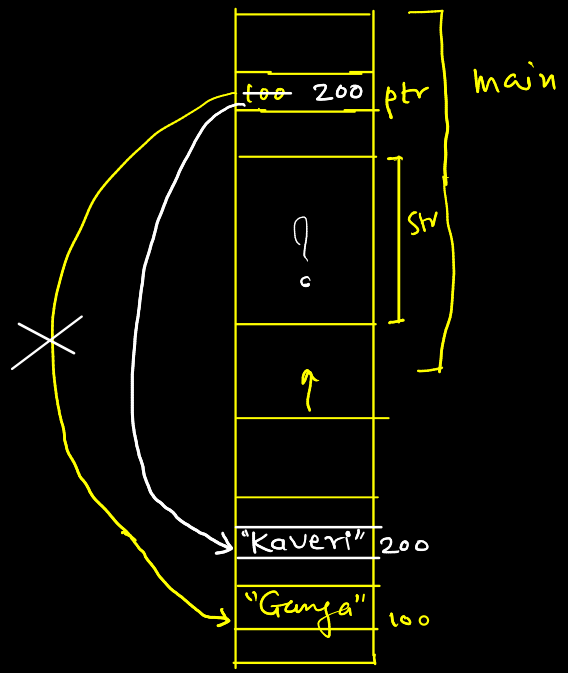
```
    // ptr is on stack, but memory
    // pointed by ptr (wher "Ganga" is
    // stored) is in code/data
    // segment and is read-only.
    // It can not be modified.
```

✗ `ptr[0] = 'S';`

```
    // Is this valid?
    char str[10];
```

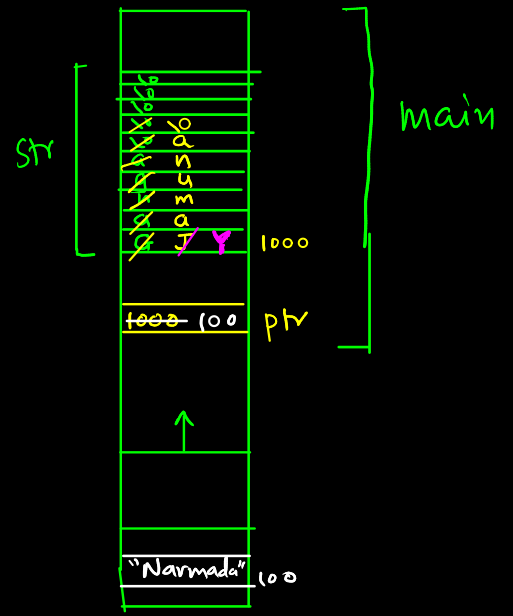
✗ `str = ptr; //No string copy`

```
    ptr = "Kaveri";
    printf("%s", ptr); // Kaveri
```



```
int main() {
   char  str[10] = "Ganga";

   printf("%s", str); // Ganga
X  //str="Saraswati";
   scanf("%s", str);

   //say input is Jamuna

   printf("%s", str); // Jamuna

   char *ptr = str;

   printf("%s", ptr); // Jamuna

   str[0] = 'Y';
   printf("%s", ptr); // Yamuna
```

such initialization is allowed only during declaration.




```
ptr = "Narmada";
printf("%s", ptr); // Narmada
printf("%s", str); // Yamuna
```

Quiz

```
Char *ptr1 = "Ram";
Char *ptr2 = "Shyam";
Char arr1[10] = "Shiv";
Char arr2[10] = "Vishnu";
```

//Are following statements valid?

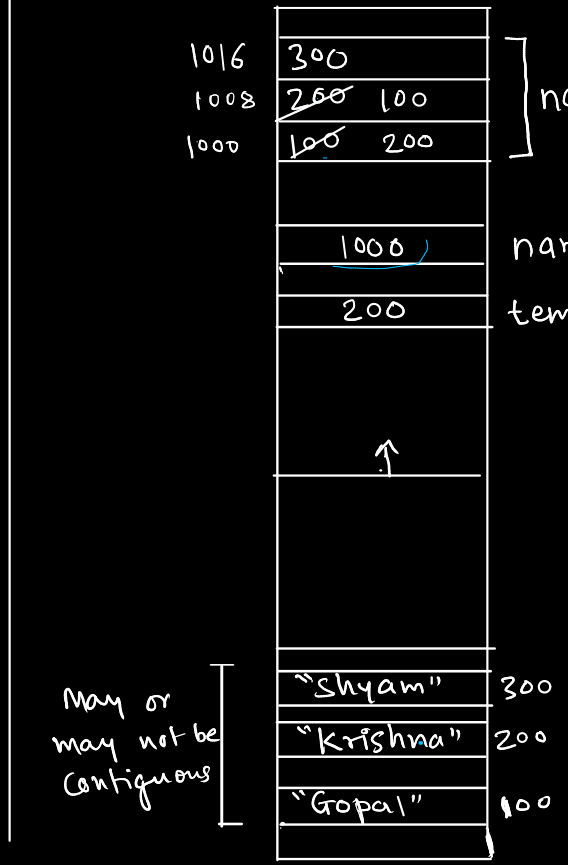
✗ `arr1 = arr2;` //arr1 is like const char *

✗ `arr1 = ptr1;` // arr1 is like const char *

✓ `ptr1 = arr1;`

✓ `ptr1 = ptr2;`

```c
#include <stdio.h>

void change(char **names) {
    char *temp = names[1];
    names[1] = names[0];
    names[0] = temp;
}

int main() {
    char *names[3] = {
                "Gopal",
                "Krishna",
                "Shyam"
            };

    change(names);

    for(int i = 0; i < 3; i++) {
        printf("%s\n", names[i]);
    }

  return 0;
}
```

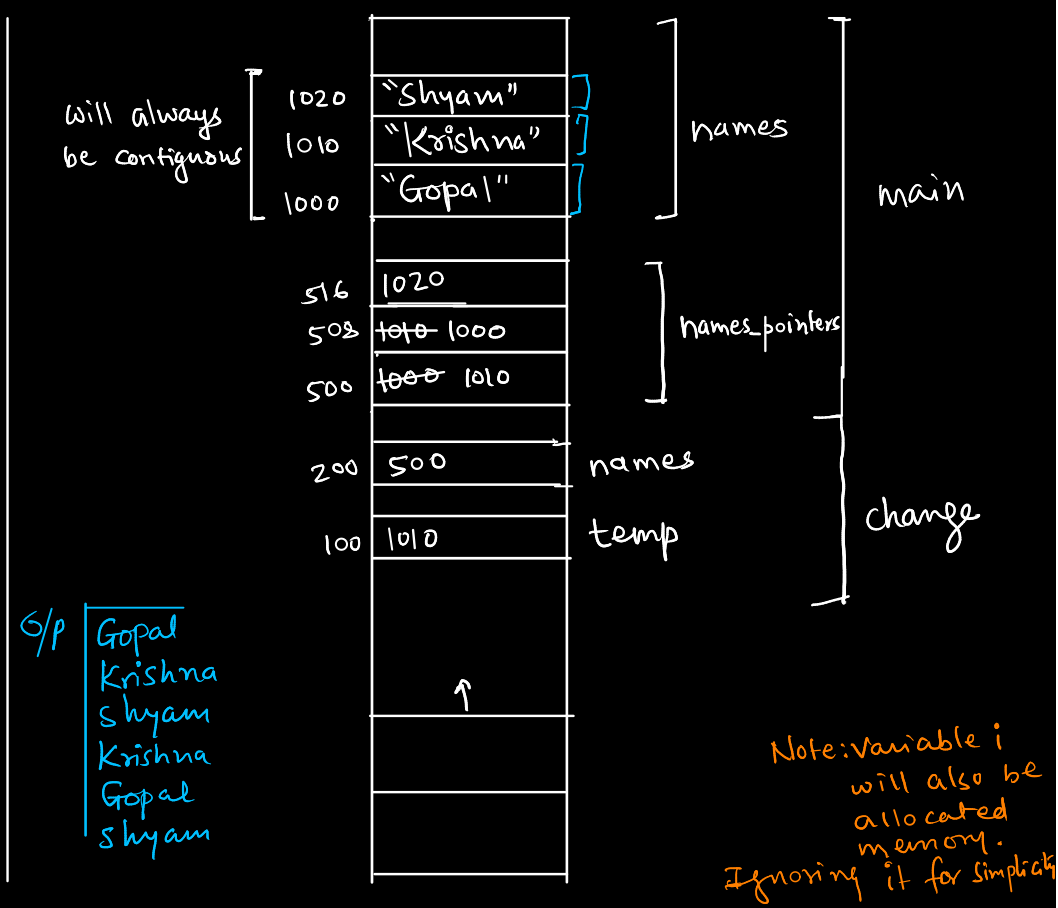


O/P
Krishna
Gopal
Shyam

names
↳ pointer to char *

Note: Variable i will also be allocated memory. Ignoring it for simplicity

```c
#include <stdio.h>
void change(char **names) {
	char *temp = names[1];
	names[1] = names[0];
	names[0] = temp;
}

int main() {
	char names[3][10] = {
				"Gopal",
				"Krishna",
				"Shyam"
			};

	char *names_pointers[3];
	for(int i = 0; i < 3; i++)
		names_pointers[i] = names[i];

	change(names_pointers);

	for(int i = 0; i < 3; i++)
		printf("%s\n", names[i]);

	for(int i = 0; i < 3; i++)
		printf("%s\n", names_pointers[i]);
	return 0;
}
```



✡Pointer to function (function pointer)

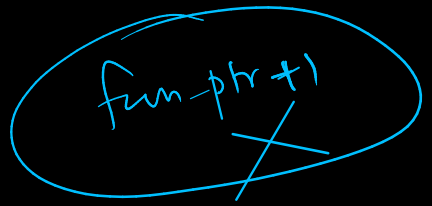
// Source: https://www.geeksforgeeks.org/function-pointer-in-c/

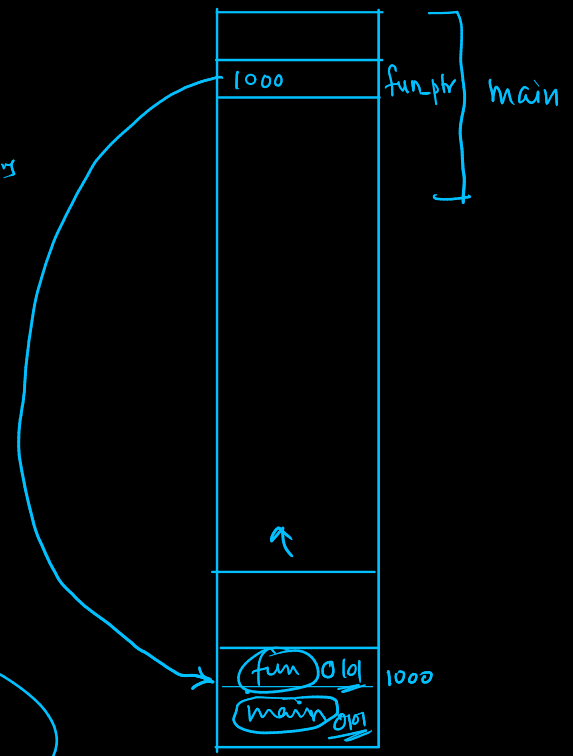
```c
#include <stdio.h>

void fun(int a) {
   printf("Value of a is %d\n", a);
}

int main() {
   // fun_ptr is a pointer to function fun()
   // Both of the following statements are valid
   // void (*fun_ptr)(int) = &fun;
      void (*fun_ptr)(int) =    fun;
   /* The above statement is equivalent of following two
      void (*fun_ptr)(int);
      fun_ptr = &fun;
   */

   // Invoking fun() using fun_ptr
   // Both of the following statements are valid
   // (*fun_ptr)(10);
   fun_ptr(10);
   return 0;
}
```

`int *nm();` // Declaration of fn returning int*

`int (*nm)();`

// Declaration of pointer to a fn returning int





Following are some interesting facts about function pointers.

1) Unlike normal pointers, a function pointer points to code, not data.
   Typically a function pointer stores the start of executable code.
2) A function's name can also be used to get functions' address.
   For example, in the above program, we have removed address operator '&' in assignment.
   We have also changed function call by removing *, the program still works.
3) No arithmetic operations can be performed on function pointer
4) Function pointer without arguments can point to any function with same return type
   int (*fptr)(); // It can point to any function returning int. Arguments can be any.
   int (*fptr2)(int);
   int fun1();
   int fun2(int);
   // Following statements are valid
   fptr = fun1; fptr();
   fptr = fun2; fptr(77);
   fptr2 = fun2; fptr2(77);
   // Following statement is not valid
   // fptr can point to a function with exactly one int argument and int return type
   fptr2 = fun1; fptr2();

```c
// A simple C program to show function pointers as parameter
#include <stdio.h>

// Two simple functions
void fun1() {
	printf("Fun1\n");
}
void fun2() {
	printf("Fun2\n");
}

// A function that receives a simple function
// as parameter and calls the function
void wrapper(void (*fun)()) {
   fun();
}

int main() {
   wrapper(fun1);
   wrapper(fun2);
   return 0;
}
```

This point in particular is very useful in C.
In C, we can use function pointers to avoid code redundancy.
For example a simple qsort() function can be used to sort
arrays in ascending order or descending or
by any other order in case of array of structures.
Not only this, with function pointers and void pointers,
it is possible to use qsort for any data type.

O/P

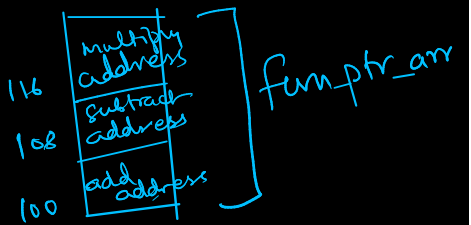
Fun1
Fun2

Many object oriented features in C++ are implemented using
function pointers in C.
For example virtual functions.
Class methods are another example implemented
using function pointers

Function pointer can be used in place of switch case.
For example, in below program, user is asked for a choice between 0 and 2 to do different tasks.

```
#include <stdio.h>
void add(int a, int b)
{
    printf("Addition is %d\n", a+b);
}
void subtract(int a, int b)
{
    printf("Subtraction is %d\n", a-b);
}
void multiply(int a, int b)
{
    printf("Multiplication is %d\n", a*b);
}
```

```
int main()
{
    // fun_ptr_arr is an array of function pointers
    void (*fun_ptr_arr[])(int, int) = {add, subtract, multiply};
    unsigned int ch, a = 15, b = 10;

    printf("Enter Choice: 0 for add, 1 for subtract and 2 "
           "for multiply\n");
    scanf("%d", &ch);

    if (ch > 2) return 0;

    (*fun_ptr_arr[ch])(a, b);

    return 0;
}
```




```
Enter Choice: 0 for add, 1 for subtract and 2 for multiply
2
Multiplication is 150
```